CS 4510 Automata and Complexity 01/15/2026

Homework 1: Finite Automata

Due:01/29/2026

This assignment is due on **11:59 PM EST, Thursday, Jan 29, 2026**. You may turn it in up to 48 hours late, but assignments turned in by the deadline receive 3% extra credit. Additionally, late submission means late feedback, which means less time to study before an exam.

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments. Enter your work using the answer environment we have provided for you, like so:

```
\begin{answer}
    <YOUR WORK GOES HERE>
\end{answer}
```

You may collaborate with other students, but any written work should be your own.

0. **Background Knowledge** (0 points)

   You should be familiar with concepts such as sets and set operations, functions, graphs, and proofs by contradiction and induction. The following problems in the book can help you review them. (Do not turn these in! We will not grade them.)

   0.2 b,e,f

   0.3 e,f

   0.5

   0.6

   0.8

   0.11

1. **Explicit Construction** ***Section A only***

   Give the state diagrams for the DFA's that accept the following languages:

   a) (4 points) $\{x | x$ is a binary string, which, when interpreted as a binary number, is equivalent to 2 *mod* 5$\}$

   Answer:

   b) (4 points) $\{x | x$ is a binary string which contains an even number of 0's and ends in 110$\}$ (Hint: language operations)

   Answer:

2. **Closure** (4 points)

Let $TWO-OF-OR-NONE(L_1, L_2, L_3)$ be the set which contains all strings which are contained in *exactly* two of the three languages or none of the languages. Show that, if L_1, L_2, and L_3 are regular, then $TWO - OF - OR - NONE(L_1, L_2, L_3)$ is also regular.

Answer:For a string to be in exactly two languages and not the third it's equivalent to the intersection of the first two languages minus the elements shared with the third. So to be in exactly L_1 and L_2 and not in L_3 we have $(L_1 \cap L_2) \setminus L_3 = L_1 \cap L_2 \cap L_3^c$. Now for all of them we have,

$$\begin{aligned} TWO - OF - OR - NONE(L_1, L_2, L_3) = &(L_1 \cap L_2 \cap L_3^c) \\ &\cup (L_3 \cap L_2 \cap L_1^c) \\ &\cup (L_1 \cap L_3 \cap L_2^c) \\ &\cup (L_1 \cup L_2 \cup L_3)^c \end{aligned}$$

Now note that if A is a regular language then A^c is also regular. Let M be the finite automata that accepts A so we have $M = (Q, \Sigma, \delta, q_0, F)$. Now note that all strings in A are such that the string ends in some accepting state in F, this means that all strings not in A i.e are in A^c end in a non-accepting state. Hence, consider the automata $M' = (Q, \Sigma, \delta, q_0, F^c)$ where the accepting states are interchanged in comparison to M. Note that now any string in A^c will end in an accepting state in M' and no string outside M' will be accepted (as they are in A and therefore by construction won't be accepted). Hence we found a finite automata that accepts A^c and therefore A^c is regular.

Now we'll show that regular languages are closed under intersections i.e. if A and B are regular then $A \cap B$ are too. We can see this by writing $A \cap B = (A^c \cup B^c)^c$. And note that A^c and B^c are regular and consequently so is $(A^c \cup B^c)$ and then $(A^c \cup B^c)^c$ using the same logic.

Now we have,

$$\begin{aligned} TWO - OF - OR - NONE(L_1, L_2, L_3) = &(L_1 \cap L_2 \cap L_3^c) \\ &\cup (L_3 \cap L_2 \cap L_1^c) \\ &\cup (L_1 \cap L_3 \cap L_2^c) \\ &\cup (L_1 \cup L_2 \cup L_3)^c \end{aligned}$$

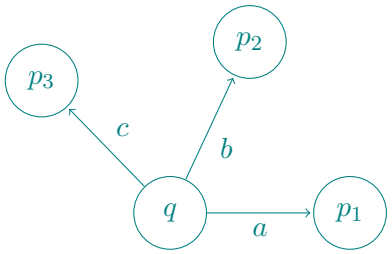
Here we have L_1, L_2, L_3 are regular. All our operations are just complements, unions and intersections. Therefore this produceses a regular language

3. **More Closure** (4 points)

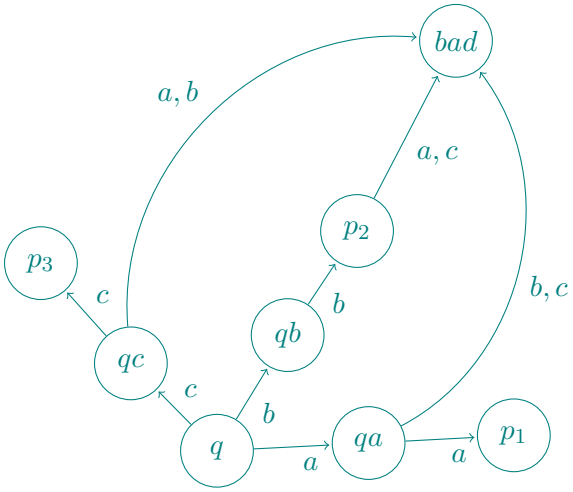
Let $L[double]$ be defined formally as $\{x_1x_1...x_nx_n | x_i \in \Sigma, x_1...x_n \in L\}$. In other words, strings of $L[double]$ are formed by taking a string from L and inserting a copy

of the previous character after each character of the word, so that if $L = \{11, 1010\}$, then $L[double] = \{1111, 11001100\}$. Prove that if L is regular, so is $L[double]$.

Answer:The goal is to construct a new automata M' that simulates the original automata M only when the same alphabet is given twice. More specifically, let the following by an arbitrary state in M,



we modify this to,



Now we define a new automata M' with the introduction of these intermediate states. An accepting state of M' is such that it corresponds to an accepting state of M . If δ' is the transition function we define it so that $\delta'(q, x_i) = q_{x_i}$ an intermediate state and we have $\delta'(q_{x_i}, x_i) = \delta(q, x_i)$ where δ is the transition function for M and $\delta'(q_{x_i}, x_j) = bad$ if $i \neq j$. So we have we have $\delta'(\delta'(q, x_i), x_i) = \delta(q, x_i)$ and if $i \neq j$ then, $\delta'(\delta'(q, x_i), x_j) = bad$. So in essence we simulate M in M' whenever we get a double, otherwise we go to bad. As we simulated M, the language that M' recognizes will be $L[double]$ as that is the condition we imposed in order to simulate M.

4. **Even More Closure** ***Section X only*** (4 points)

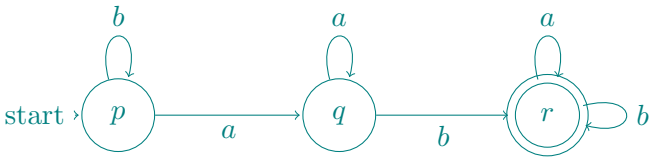
   Let $END(L) = \{x \mid \text{ for some } y \in \Sigma^*, yx \in L\}$. Thus $END(L)$ is the set of strings which could appear as a suffix of a string in L. Prove that if L is regular, $END(L)$ is regular.

Answer:Note that for any suffix of a string $x_1 \dots x_n$, $x_i \dots x_n$, there exists a prefix of $REVERSE(x) = x_n \dots x_1$ of the form $x_n \dots x_i$ that is the reverse of the suffix.

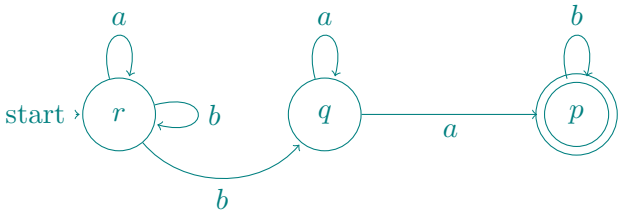
Point is, it is enough to show that regular languages are closed under reversal and closed under taking prefixes and we can write our problem as,

$$END(L) = REVERSE(Prefix(REVERSE(L)))$$

So first we need to show that if L is regular than $REVERSE(L)$ is regular as well. We can use a NFA first. Let the DFA associated with L be M. Now we convert the start state of M to be an accept state and have the various accept states of M to be start states (possible as we're using an NFA). Now we reverse each transition as well. For instance if we have the following DFA,

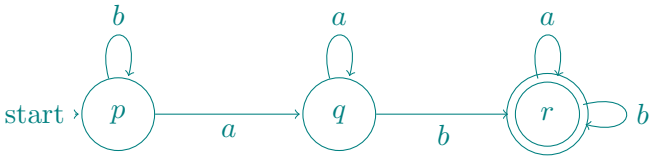


we construct the NFA as follows,

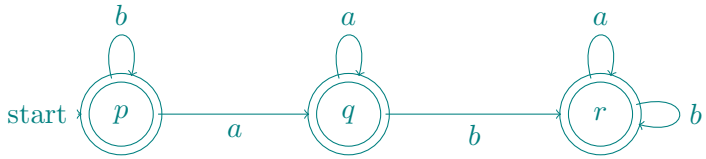


So in our new construction of M' if δ' is the transition function and δ is of M, then if $\delta(p, a) = q$ we have $\delta'(q, a) = p$. Now note that the produced DFA from this accepts only the reverse language L and hence we have M' is regular.

Now we just need to show that given a language L with automaton M we can construct a automaton M' that accepts that language L along with all it's prefixes. To do this, all we need to do is find every path to an accept state and for all states along that path we modify it to be a accept state itself. For instance we have,



modified to,



If q_0 is the start state and p is the accepting state, then we define the states in a path to be all states $q_1, \ldots, q_n$ (repetitions allowed) such that we have $\delta(q_n, x_n) = p$ and $\delta(q_i, x_i) = q_{i+1}$ for $i < n$ where $x_1 \ldots x_n$ are alphabets (repetitions allowed). So then we have all q_i where $i = 1, \ldots, n$ to be accepting. (So for instance if an accepting state points to another state from which there is no path back to an accepting state, then it won't be marked accepting).

So this proves that the set of prefixes of a language is also regular. Hence we have $REVERSE(L)$ is regular, so $Prefix(REVERSE(L))$ is regular and lastly $REVERSE(Prefix(REVERSE(L)))$ is also regular and we showed that this is equivalent to $SUFFIX(L)$.

5. **Minimal DFA's** ***Section X only*** (4 points)

Let L_k be the regular language over $\{0, 1\}$ which contains all strings which have a 1 as the k'th character from the right end of the string. Prove that any DFA that recognizes L_k must contain at least 2^k states.

Answer:First let us assume there exists a DFA that has less than 2^k states and is able to recognize whether there is a 1 in the $k'th$ character from the right end of the string. Now note that there are 2^k different distinct strings of length k. Let us consider the ordering from smallest to highest. For each string being computed by our automata, let the string end at state q_i where i is the position of our string in our lowest-highest ordering. So we have,

$$\begin{aligned} 000\ldots0 &\to q_0 \\ 000\ldots1 &\to q_1 \\ &\vdots \\ 111\ldots1 &\to q_{2^k-1} \end{aligned}$$

Now clearly we have defined q_i for each string and hence we have 2^k states defined above, however by assumption we have less than 2^k states in our automata, hence there must be at least a pair of strings say x_i, x_j such that they have the same state i.e $q_i = q_j$. Now we know that $x_i \neq x_j$ so there must be some position in the string where they differ. More specifically, WLOG let x_i contain a 1 in the l'th position from the right and x_j contain a 0 in the l'th position from the right as well. So we have for example,

$$\begin{aligned} x_i = *\ldots1\cdots* &\to q_i = q_j \\ x_j = *\ldots0\cdots* &\to q_j = q_i \end{aligned}$$

where $*$ denotes an arbitrary value of either 0 or 1.

Now let us add $k - l + 1$ zeroes to the rightmost side for both strings to get,

$$1 \cdots * 0 \ldots 0$$
$$0 \cdots * 0 \ldots 0$$

Now note that with respect to the new string, the l'th position ends at $q_i = q_j$ for both strings, however as now we added a bunch of zeroes, the computation is equivalent for both strings i.e. the new state q_{i0} of the new string would be equivalent to q_{j0} of the second string. But note that both strings are of length k and string 1 starts with a 1 while string 2 starts with a 0. By assumption our DFA accept string 1 and rejects string 2, so we cannot have both strings end at the same state as that would mean either both are accepting or both are rejecting, a contradiction. Hence, our assumption must have been wrong and there is no DFA with fewer than 2^k states that can solve our problem